

| 操作系统作业2

实验环境：Ubuntu22.04.5

语言：C

| 父进程被撤销后，子进程也一并被撤销

| (1) 父进程在退出前主动杀掉子进程

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

int main() {
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        exit(1);
    }
    else if(pid == 0) {
        // 子进程：持续运行
        while(1) {
            sleep(1);
            printf("Child: Running.\n");
        }
        exit(0);
    }
    else {
        // 父进程：等待2秒后主动杀掉子进程再退出
        printf("Parent: I am alive, child pid = %d\n", pid);
        sleep(2);
        printf("Parent: Killing child and exiting.\n");
        kill(pid, SIGKILL);
        exit(0);
    }
}
```

我们发现子进程已经结束

```

(base) root@REFINE-LV:~# vim pro_1.c
(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      36625   36417   0 18:08 pts/0    00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~# gcc pro_1.c -o pro_1
^[[A(base) root@REFINE-LV:~# ./pro_1
Parent: I am alive, child pid = 36703
Child: Running.
Parent: Killing child and exiting.
(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      36725   36417   0 18:08 pts/0    00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~# |

```

I (2) 子进程使用“父死亡信号”机制 (PDEATHSIG)

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/prctl.h>
#include <signal.h>

void handle_signal(int sig) {
    printf("Child: Received signal %d, exiting.\n", sig);
    exit(0);
}

int main() {
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        exit(1);
    }
    else if(pid == 0) {
        // 子进程: 设置父死亡信号
        if (prctl(PR_SET_PDEATHSIG, SIGTERM) != 0) {
            perror("prctl");
            exit(1);
        }

        // 防止子进程运行时父进程已经退出 (竞态)
        if (getppid() == 1) {
            printf("Child: Parent is already dead. Exiting.\n");
            exit(0);
        }

        signal(SIGTERM, handle_signal); // 捕获 SIGTERM 信号
        while(1) {
            sleep(1);
        }
    }
}

```

```

        printf("Child: Running. My current parent is %d\n", getppid());
    }
}
else {
    // 父进程
    printf("Parent: I am alive, child pid = %d\n", pid);
    sleep(3);
    printf("Parent: Exiting now.\n");
    exit(0);
}
}

```

我们发现子进程已经结束

```

(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      32803   32772  0 18:00 pts/0    00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~# ./pro_1
Parent: I am alive, child pid = 32860
Child: Running. My current parent is 32859
Child: Running. My current parent is 32859
Parent: Exiting now.
Child: Received signal 15, exiting.
(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      32884   32772  0 18:00 pts/0    00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~# |

```

I 父进程退出后子进程继续存在

一般情况，父进程退出后，子进程会自动成为孤儿进程，被 init（或 systemd）收养并继续运行。

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();
    if(pid < 0) {
        perror("fork");
        exit(1);
    }
    else if(pid == 0) {
        //子进程
        sleep(5);
        printf("Child: After sleep, my parent is %d\n", getppid());
    }
}

```

```

        sleep(3600);
        exit(0);
    }
    else {
        //父进程
        printf("Parent: I am exiting, child pid = %d\n", pid);
        exit(0);
    }
}

```

我们发现当父进程结束的时候，子进程被init接管

```

(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      34658   34095   0 18:04 pts/3      00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~# gcc pro_1.c -o pro_1
(base) root@REFINE-LV:~# ./pro_1
Parent: I am exiting, child pid = 34695
(base) root@REFINE-LV:~#
(base) root@REFINE-LV:~# Child: After sleep, my parent is 34091

(base) root@REFINE-LV:~# ps -ef | grep pro_1
root      34695   34091   0 18:04 pts/3      00:00:00 ./pro_1
root      34795   34095   0 18:04 pts/3      00:00:00 grep --color=auto pro_1
(base) root@REFINE-LV:~#

```

I 总结

从上述分析可以归纳出，在 Linux 系统中，父进程退出后对子进程行为的影响主要取决于子进程自身的设置和所属环境管理策略，具体包括：

- **父进程退出同时终止子进程的情况：**
 1. 子进程设置了父死亡信号（PDEATHSIG）。
 2. 父进程退出前主动发送信号杀掉子进程。
- **父进程退出后子进程继续存在的情况：**
 1. 子进程未设置任何与父进程终止相关的处理，默认成为孤儿进程，被 init（或 systemd）收养。
 2. 子进程主动脱离终端或会话（如调用 `setsid()`），使其不受 SIGHUP 信号影响，从而继续运行。